

Enterprise AI Knowledge Assistant

Hybrid OKF + RAG Development Plan

Project Vision

Build an enterprise AI knowledge assistant that uses:

- **OKF (Open Knowledge Format)** for structured enterprise knowledge
 - **RAG (Retrieval-Augmented Generation)** for unstructured documents
 - **Hybrid Retrieval** to decide whether to use OKF, RAG, or both
 - **Citations and sources** in every answer
-

Final System Flow

1. Admin uploads a document or structured record.
 2. System extracts text and metadata.
 3. System converts suitable facts into OKF objects.
 4. System chunks documents for RAG.
 5. System stores chunks in a vector database.
 6. User asks a question.
 7. Query classifier decides:
 8. OKF only
 9. RAG only
 10. OKF + RAG together
 11. Retrieved context is passed to the LLM.
 12. LLM generates grounded answer with citations.
-

Phase 0 — Planning and Architecture

Goal

Finalize the product idea, scope, and architecture before writing code.

Exact Things to Do

1. Write the project problem statement.

2. Define the target users:
3. Admin
4. Employee/User
5. List supported knowledge types:
6. Policies
7. FAQs
8. Documents
9. Departments
10. Employees
11. Products
12. Rules
13. Decide what goes to OKF and what goes to RAG.
14. Draw the high-level architecture diagram.
15. Define the data flow from upload to answer.
16. Finalize the technology stack.
17. Create the folder structure for backend and frontend.
18. Write the minimum viable product scope.
19. Prepare the phase-wise learning plan for the team.

Deliverables

- Project synopsis
 - HLD diagram
 - System flow diagram
 - Folder structure
 - Tech stack list
 - MVP scope document
-

Phase 1 — Backend Foundation

Goal

Set up the backend, database, and basic APIs.

Exact Things to Do

1. Create the FastAPI project.
2. Set up virtual environment and dependencies.
3. Create configuration files for environment variables.

4. Connect PostgreSQL to the backend.
5. Create SQLAlchemy models.
6. Set up Alembic migrations.
7. Create authentication APIs:
8. Signup
9. Login
10. Token generation
11. Token verification
12. Create user roles:
13. Admin
14. Employee
15. Create base API structure:
16. `users`
17. `auth`
18. `documents`
19. `knowledge`
20. `chat`
21. Add error handling and logging.
22. Create health check endpoint.
23. Test all APIs with Swagger/Postman.

Deliverables

- Working FastAPI backend
- PostgreSQL connection
- JWT-based login system
- Role-based access foundation
- API documentation

Phase 2 — Document Management

Goal

Allow admins to upload, store, view, update, and delete documents.

Exact Things to Do

1. Create document upload API.
2. Allow file types:
 3. PDF
 4. DOCX
 5. PPTX
 6. TXT
 7. Markdown
8. Store uploaded files in local storage or cloud storage.
9. Save document metadata in database.
10. Create document listing API.
11. Create document delete API.
12. Create document update API.
13. Create document status field:
 14. Uploaded
 15. Processing
 16. Ready
 17. Failed
18. Add file validation:
 19. Size limit
 20. Type check
21. Create a document history/version field.

Deliverables

- Document upload module
 - Document CRUD APIs
 - Document metadata table
 - Upload status tracking
-

Phase 3 — Text Extraction and Cleaning

Goal

Extract clean text from uploaded documents.

Exact Things to Do

1. Build PDF text extraction.
2. Build DOCX text extraction.
3. Build PPTX text extraction.
4. Build TXT and Markdown reading.
5. Add OCR support for scanned PDFs.
6. Remove headers, footers, and extra spaces.
7. Normalize line breaks.
8. Remove unwanted symbols and noise.
9. Detect empty or broken documents.
10. Save extracted raw text in database or file storage.
11. Show extraction status in the admin dashboard.

Deliverables

- Text extraction pipeline
 - OCR support
 - Clean text output
 - Extraction status API
-

Phase 4 — Chunking and Metadata Preparation

Goal

Break documents into meaningful chunks for retrieval.

Exact Things to Do

1. Split extracted text into chunks.
2. Decide chunk size and overlap.
3. Try multiple chunking strategies:
 4. Fixed-size chunking
 5. Sentence-based chunking
 6. Section-based chunking
7. Attach metadata to each chunk:
 8. document_id
 9. page number

10. section title
11. upload date
12. source file name
13. Store chunks in the database.
14. Create chunk preview API.
15. Add chunk status tracking.

Deliverables

- Chunking service
 - Chunk metadata table
 - Chunk preview endpoint
 - Chunk storage logic
-

Phase 5 — OKF Knowledge Extraction

Goal

Convert important structured facts from documents into OKF objects.

Exact Things to Do

1. Define the OKF schema for the project.
2. Decide the object types:
3. Policy
4. Employee
5. Department
6. Product
7. FAQ
8. Business rule
9. Asset
10. Create a structured JSON format for knowledge objects.
11. Build entity extraction logic.
12. Build relation extraction logic.
13. Extract important facts from documents.
14. Validate extracted facts against schema.
15. Save knowledge objects in the database.
16. Link each OKF object back to its source document.
17. Create CRUD APIs for OKF objects.
18. Add search API for structured knowledge.
19. Add update and versioning support.

Deliverables

- OKF schema design
 - Structured knowledge table
 - Knowledge extraction pipeline
 - Knowledge CRUD APIs
 - Source traceability for every object
-

Phase 6 — Vector Embeddings and Semantic Search

Goal

Enable semantic retrieval from unstructured document chunks.

Exact Things to Do

1. Choose an embedding model.
2. Generate embeddings for every chunk.
3. Store embeddings in a vector database.
4. Save chunk ID with each vector.
5. Add metadata filtering.
6. Build semantic search API.
7. Return top-k relevant chunks for a query.
8. Add similarity score to results.
9. Test retrieval for different user queries.

Deliverables

- Embedding generation service
 - Vector database integration
 - Semantic search API
 - Top-k retrieval results
-

Phase 7 — Query Classification and Hybrid Retrieval

Goal

Decide whether to use OKF, RAG, or both for a query.

Exact Things to Do

1. Build query intent classifier.
2. Classify questions into:
3. Structured query
4. Document query
5. Mixed query
6. Use OKF retrieval for facts, rules, and entities.
7. Use RAG retrieval for long-form document context.
8. Use both when the question needs both types.
9. Merge results from OKF and RAG.
10. Remove duplicate or low-quality context.
11. Rank retrieved information by relevance.
12. Pass final context to prompt builder.

Example Rules

- "Who is the HR manager?" → OKF
- "Summarize the leave policy" → RAG
- "Explain the leave policy and tell me the carry-forward limit" → OKF + RAG

Deliverables

- Query classifier
 - Hybrid retriever
 - Context merger
 - Retrieval ranking logic
-

Phase 8 — LLM Answer Generation

Goal

Generate grounded answers from retrieved knowledge.

Exact Things to Do

1. Build prompt template.
2. Add system instructions for grounded answers.
3. Insert retrieved OKF and RAG context into the prompt.
4. Ask the LLM to answer only from provided context.
5. Add citation format in the response.

6. Add fallback response when context is insufficient.
7. Add structured output if needed.
8. Return answer with source references.
9. Add confidence or relevance score.

Deliverables

- Prompt builder
 - Answer generation service
 - Citation support
 - Fallback response logic
-

Phase 9 — Frontend Development

Goal

Create the user interface for admin and employee users.

Exact Things to Do

1. Create login and signup pages.
2. Create admin dashboard.
3. Create document upload screen.
4. Create document list and processing status view.
5. Create OKF knowledge management page.
6. Create chat interface for employees.
7. Show retrieved sources in chat.
8. Show related documents and structured facts.
9. Add loading and error states.
10. Make the UI responsive.

Deliverables

- Admin portal
 - Employee portal
 - Chat interface
 - Source display panel
-

Phase 10 — Security and Access Control

Goal

Protect enterprise knowledge with proper permissions.

Exact Things to Do

1. Implement JWT authentication.
2. Implement OAuth2 if needed.
3. Add role-based access control.
4. Restrict document access by role or department.
5. Restrict OKF objects by permission.
6. Add audit logs for uploads, edits, and queries.
7. Secure all APIs.
8. Validate user permissions before retrieval.
9. Hide unauthorized sources from results.

Deliverables

- Secure login system
 - RBAC rules
 - Permission checks
 - Audit logs
-

Phase 11 — Production Features

Goal

Make the system faster, smarter, and more stable.

Exact Things to Do

1. Add Redis caching.
2. Add background indexing jobs.
3. Add automatic re-indexing when documents change.
4. Add hybrid search improvements.
5. Add reranking for better results.
6. Add feedback collection.
7. Add query history.
8. Add usage analytics.
9. Add monitoring and structured logging.
10. Add performance testing.

Deliverables

- Cached retrieval
 - Background jobs
 - Search analytics
 - Performance monitoring
-

Phase 12 — Deployment and DevOps

Goal

Deploy the system like a real enterprise product.

Exact Things to Do

1. Dockerize backend.
2. Dockerize frontend.
3. Create Docker Compose file.
4. Configure environment variables.
5. Set up NGINX if needed.
6. Create GitHub Actions pipeline.
7. Add production build steps.
8. Add health checks.
9. Add backup strategy.
10. Deploy to cloud server.

Deliverables

- Docker setup
 - CI/CD pipeline
 - Deployment guide
 - Production-ready release
-

Final MVP Checklist

The first working version should do all of the following: 1. Admin uploads a PDF. 2. System extracts text. 3. System chunks the text. 4. System generates embeddings. 5. System stores chunks in vector DB. 6. System extracts structured OKF objects. 7. User asks a question. 8. Query classifier selects OKF, RAG, or both. 9. System retrieves relevant knowledge. 10. LLM generates grounded answer. 11. Answer shows source citations.

Recommended Order of Work

1. Planning and architecture
 2. Backend foundation
 3. Document upload
 4. Text extraction
 5. Chunking
 6. OKF extraction
 7. Embeddings and vector search
 8. Hybrid retrieval
 9. LLM answer generation
 10. Frontend
 11. Security
 12. Production features
 13. Deployment
-

Team Rule

Do not move to the next phase until the current phase is stable, tested, and documented.